Nom i Cognom:	**Anna Puig Mosquera**
Enllaç al repositori Git:	https://github.com/AnnSan00/AMS2-0486-RA1-PR1.4-puiga.git

Objectius:

- Aprendre a llegir i escriure fitxers JSON amb Java (Jakarta JSON-P i Jackson 3).

Instruccions:

- Desenvolupa el codi necessari per a cada exercici seguint les especificacions indicades. El codi ha de superar els tests proporcionats (mvn test) per garantir-ne la funcionalitat.
- En cas d'haver de contestar alguna pregunta en aquest document, cal fer-ho dins del quadre indicat.

Criteris d'avaluació:

Cada exercici té especificada la puntuació. Es valorarà:

- Format correcte del codi (indentació i compliment de la guia d'estil de Java).
- Noms clars i descriptius per a mètodes i variables.
- Qualitat general del codi, amb comentaris explicatius quan sigui necessari.
- Gestió correcta de les excepcions (fitxer inexistent o JSON no vàlid).
- Les còpies seran penalitzades amb una puntuació de 0.

Entrega:

- Repositori Git privat, compartit amb l'usuari jpala4-ieti, que contingui el codi complet de la pràctica.
- Subdirectori "doc" dins del repositori amb el fitxer memoria.pdf.
- URL del repositori: S'ha de pujar a Moodle.

Recursos i materials:

- JDK 25 (LTS) o, com a mínim, JDK 21; Maven 3.9 o superior i Git instal·lats.
- Eina de programació: necessiteu un IDE per programar en Java (VS Code, IntelliJ IDEA o un altre).
- Altres recursos: documentació oficial i tutorials. Si trobeu exemples de Jackson 2 (com.fasterxml.jackson), adapteu-los a Jackson 3 (tools.jackson).

Punt de partida de la pràctica:

https://github.com/jpala4-ieti/DAM-M0486-RA1-PR1.4-Practica-Punt-Partida-26-27

Introducció

Context:
Ets el desenvolupador d'una petita llibreria digital. Les dades dels llibres estan emmagatzemades en un fitxer JSON anomenat llibres_input.json (directori data/pr14). Cada llibre té un títol, un autor, un any de publicació i un identificador únic.

Contingut del fitxer llibres_input.json:

```
[
  {"id": 1, "titol": "Els vents del nord", "autor": "Clara Rojas", "any": 1990},
  {"id": 2, "titol": "Muntanyes màgiques", "autor": "Joan Peris", "any": 2005},
  {"id": 3, "titol": "El secret del mar", "autor": "Laura Vidal", "any": 2010}
]
```

Contingut esperat dels fitxers de sortida després de l'execució del programa:

```
[
  {"id": 1, "titol": "Els vents del nord", "autor": "Clara Rojas", "any": 1995},
  {"id": 3, "titol": "El secret del mar", "autor": "Laura Vidal", "any": 2010},
  {"id": 4, "titol": "Històries de la ciutat", "autor": "Miquel Soler", "any": 2022}
]
```

Nota: l'ordre de les claus dins de cada objecte depèn de la biblioteca i de la versió que es faci servir (per exemple, Jackson 3 les escriu per ordre alfabètic per defecte). En JSON aquest ordre no és significatiu: si el vostre fitxer mostra les claus en un ordre diferent del d'aquest exemple, no tindrà cap impacte negatiu en la nota. El que ha de coincidir són les dades.

Revisa i prova el codi dels exemples:
https://github.com/jpala4-ieti/DAM-M0486-RA1-PR1.4-Practica-Punt-Partida-26-27/tree/main/src/main/java/com/project/exemples

Exercici 0: Gestió d'una llibreria digital amb JSON-P (4 punts)

Modifica el programa `PR14GestioLlibreriaJakartaMain.java`
Referència:
https://github.com/jpala4-ieti/DAM-M0486-RA1-PR1.4-Practica-Punt-Partida-26-27/blob/main/src/main/java/com/project/exemples/GestioPersonesJakartaMain.java

Tasques a realitzar:

- **Lectura del fitxer:** Llegeix el fitxer `llibres_input.json` i carrega'l a una estructura de dades en memòria.
- **Modificació:** Canvia l'any de publicació del llibre amb id 1 a 1995.

- **Afegit:** Afegeix un nou llibre amb id 4, títol "Històries de la ciutat", autor "Miquel Soler", any 2022.
- **Esborrat:** Esborra el llibre amb id 2.
- **Guardat:** Guarda les dades modificades en un fitxer nou anomenat `llibres_output_jakarta.json`, al mateix directori que el fitxer d'entrada.
- **Gestió d'errors:** si el fitxer no existeix o no conté un JSON vàlid, `carregarLlibres()` ha de mostrar un missatge d'error i retornar `null`, sense propagar cap excepció.

Comprova la teva solució amb: `mvn test -Dtest=PR14GestioLlibreriaJakartaMainTest`

Exercici 1: Gestió d'una llibreria digital amb Jackson 3 (4 punts)

Modifica el programa `PR14GestioLlibreriaJacksonMain.java`
Referència:
https://github.com/jpala4-ieti/DAM-M0486-RA1-PR1.4-Practica-Punt-Partida-26-27/blob/main/src/main/java/com/project/exemples/GestioPersonesJacksonMain.java
Important: fem servir Jackson 3 (paquets `tools.jackson.*`). Les seves excepcions (`JacksonException`) són no comprovades (unchecked): el compilador no t'obligarà a capturar-les, però les has de gestionar igualment.

Tasques a realitzar:

- **Lectura del fitxer:** Llegeix el fitxer `llibres_input.json` i carrega'l a una estructura de dades en memòria.
- **Modificació:** Canvia l'any de publicació del llibre amb id 1 a 1995.
- **Afegit:** Afegeix un nou llibre amb id 4, títol "Històries de la ciutat", autor "Miquel Soler", any 2022.
- **Esborrat:** Esborra el llibre amb id 2.
- **Guardat:** Guarda les dades modificades en un fitxer nou anomenat `llibres_output_jackson.json`, al mateix directori que el fitxer d'entrada.
- **Gestió d'errors:** si el fitxer no existeix o no conté un JSON vàlid, `carregarLlibres()` ha de mostrar un missatge d'error i retornar `null`, sense propagar cap excepció.

Comprova la teva solució amb: `mvn test -Dtest=PR14GestioLlibreriaJacksonMainTest`

Exercici 2 (2 punts)

Explica a la memòria (doc/memoria.pdf) què fan els programes:

- https://github.com/jpala4-ieti/DAM-M0486-RA1-PR1.4-Practica-Punt-Partida-26-27/blob/main/src/main/java/com/project/exemples/ExempleClient.java
- https://github.com/jpala4-ieti/DAM-M0486-RA1-PR1.4-Practica-Punt-Partida-26-27/blob/main/src/main/java/com/project/exemples/ExempleServidor.java

Després, modifica el servidor perquè cada petició que rebi quedi emmagatzemada com a fitxer JSON dins del directori `data/requests`:

- Cada petició s'ha de guardar en un fitxer diferent (per exemple, amb la data i l'hora al nom). Si el directori no existeix, s'ha de crear.
- El fitxer ha de ser un JSON vàlid i ha d'incloure, com a mínim, la data i l'hora, el mètode HTTP i el cos de la petició.
- Si no es pot guardar la petició, el servidor ha de continuar responent al client.
- Afegeix a la memòria el contingut d'algun dels fitxers generats.

ExempleClient.java: Aquest programa s'encarrega d'enviar una petició HTTP (com ara un mètode GET o POST) cap a un servidor local, enviant determinades dades o paràmetres i rebent la resposta corresponent per mostrar-la per consola.
ExempleServidor.java: Aquest programa aixeca un servidor HTTP bàsic en Java (utilitzant la classe HttpServer) que queda a l'espera de rebre peticions de clients, processar-les i retornar una resposta HTTP amb un missatge de confirmació.
Modificacions implementades al servidor S'ha integrat un mecanisme perquè el servidor arxivés cada sol·licitud rebuda dins del directori data/requests en format JSON independent. Si la carpeta no existeix prèviament, el codi s'encarrega de crear-la de manera autònoma. Per evitar que els fitxers es trepitgin entre ells, el nom s'assigna combinant la data i l'hora exacta amb precisió temporal.
En cada document emmagatzemat queda constància del moment precís de l'arribada, el verb HTTP utilitzat i la càrrega útil enviada. S'ha previst a més a més una protecció de control d'errors: si per qualsevol motiu el disc falla al desar el fitxer, el servidor ho notifica internament però no s'atura, permetent que el client continuï rebent la seva resposta de manera habitual.

